Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ***ИУК «Информатика и управление»***

КАФЕДРА ***ИУК5 «Системы обработки информации»***

Лабораторная работа № 2

«Регрессия»

ДИСЦИПЛИНА: «Технологии обработки больших данных»

Выполнил: студент гр.ИУК5-82Б _____ (Ли Р.В. _____)
(Подпись) (Ф.И.О.)

Проверил: _____ (_____)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

Калуга, 2026

Цель лабораторной работы - освоить полный пайплайн решения задачи регрессии на табличных данных: анализ данных, предобработка, обучение нескольких моделей, подбор гиперпараметров, сравнение качества и интерпретация результатов.

Задачи:

- провести первичный анализ набора данных и выявлять проблемы качества данных;
- построить воспроизводимый пайплайн предобработки с использованием Pipeline и ColumnTransformer;
- обучить и сравнить несколько регрессионных моделей по метрикам MAE, RMSE, R^2;
- выполнить подбор гиперпараметров (GridSearchCV или RandomizedSearchCV);
- визуализировать результаты и интерпретировать модель.

Для выполнения работы могут быть использованы следующие варианты датасетов:

- Diamonds (Kaggle): shivam2503/diamonds. Целевая переменная - price (стоимость бриллианта).
- California Housing (sklearn). Целевая переменная: MedHouseVal по признакам района.
- свой датасет (по согласованию с преподавателем).

Для тренировки можно использовать руководство с использованием датасета Kaggle hellbuoy/car-price-prediction.

Задание для лабораторной работы

Выполните пункты 1 -2 в одном ноутбуке. Все результаты (таблицы/графики/выводы) должны присутствовать в ноутбуке.

1. *Загрузка и первичный анализ*
   - Загрузите файл данных.
   - Выведите размерность (shape), первые строки (head), типы данных (dtypes).
   - Проверьте пропуски (isna().sum()) и дубликаты (duplicated().sum()).
   - Проанализируйте распределение целевой переменной (гистограмма, boxplot).
2. *Разделение данных*

   Разделите данные на обучающую и тестовую выборки (train/test) с фиксированным random_state. Целевую переменную вынесите в отдельный вектор y.
3. *Предобработка (обязательно через Pipeline)*

Постройте единый пайплайн предобработки с использованием ColumnTransformer:

- численные признаки: заполнение пропусков (например, медианой) + масштабирование (StandardScaler);
- категориальные признаки: заполнение пропусков (например, most_frequent) + кодирование (OneHotEncoder(handle_unknown='ignore')).

Пайплайн должен быть воспроизводимым и применяться одинаково к train и test.

4. *Обучение базовых моделей*

Обучите и сравните минимум 4 модели:

- LinearRegression (baseline);
- Ridge или Lasso (регуляризация);
- KNeighborsRegressor;
- DecisionTreeRegressor;

Дополнительно (рекомендуется): RandomForestRegressor или GradientBoostingRegressor.

5. *Подбор гиперпараметров*

Выполните подбор гиперпараметров минимум для двух моделей (например, Ridge и KNN, или KNN и RandomForest) с использованием GridSearchCV либо RandomizedSearchCV. Используйте кросс-валидацию (cv=5).

6. *Оценка качества*

Для каждой модели посчитайте на тестовой выборке метрики:

- MAE
- RMSE
- R^2

Сведите результаты в единую таблицу (DataFrame) и отсортируйте по RMSE (по возрастанию).

7. *Визуализация результатов*

Постройте графики для лучшей модели:

- точечная диаграмма y_true vs y_pred;
- гистограмма остатков (y_true - y_pred);
- график важности признаков/коэффициентов (если применимо: top -10).

8. *Итоговые выводы*

Сформулируйте выводы: какая модель показала лучший результат и почему; какие признаки оказывают наибольшее влияние; какие проблемы данных были обнаружены и как их учитывали в предобработке.

Выполнение:

1. Загрузка и первичный анализ

- Загрузите файл данных.
- Выведите размерность (shape), первые строки (head), типы данных (dtypes).
- Проверьте пропуски (isna().sum()) и дубликаты (duplicated().sum()).
- Проанализируйте распределение целевой переменной (гистограмма, boxplot).

```
[3]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     import seaborn as sns

[5]: df = pd.read_csv("diamonds.csv")

[6]: df.shape

[6]: (53940, 11)

[7]: df.head()
```

[7]:

	Unnamed: 0	carat	cut	color	clarity	depth	table	price	x	y	z
0	1	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	2	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	3	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	4	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	5	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

```
[8]: df.dtypes

[8]: Unnamed: 0      int64
     carat         float64
     cut               str
     color             str
     clarity           str
     depth         float64
     table         float64
     price           int64
     x             float64
     y             float64
     z             float64
     dtype: object

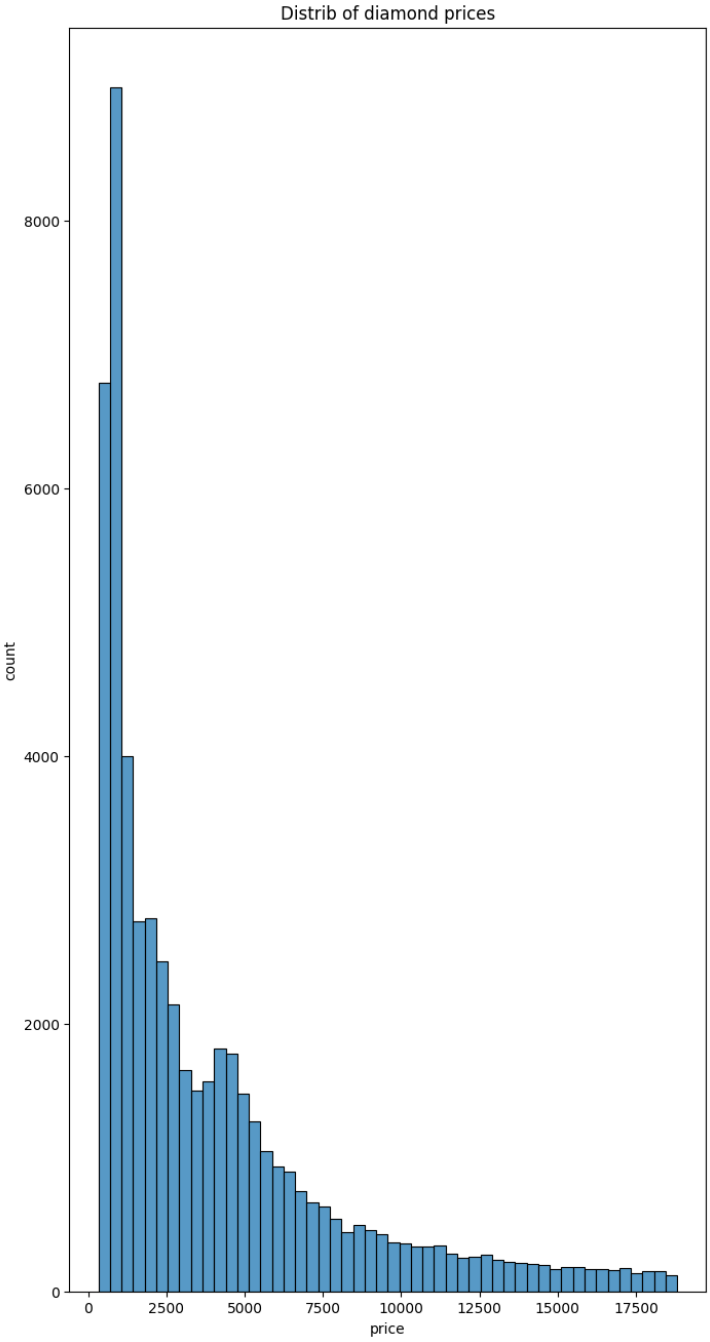
[9]: df.isna().sum()

[9]: Unnamed: 0    0
     carat         0
     cut           0
     color         0
     clarity       0
     depth         0
     table         0
     price         0
     x             0
     y             0
     z             0
     dtype: int64

[10]: df.duplicated().sum()

[10]: np.int64(0)
```

[16]:
```
plt.figure(figsize=(8, 16))
sns.histplot(df['price'], bins=50)
plt.title("Distrib of diamond prices")
plt.xlabel("price")
plt.ylabel("count")
plt.show()
```



2. Разделение данных

Разделите данные на обучающую и тестовую выборки (train/test) с фиксированным random_state. Целевую переменную вынесите в отдельный вектор y.

[10]:
```
X = df.drop('price', axis = 1)
y = df['price']
```

[11]:
```
print(X.shape)
print(y.shape)
```

```
(53940, 10)
(53940,)
```

[12]:
```
from sklearn.model_selection import train_test_split
```

[13]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

[14]:
```
print("train set:", X_train.shape)
print("test set:", X_test.shape)
```

```
train set: (43152, 10)
test set: (10788, 10)
```

3. Предобработка (обязательно через Pipeline)

Постройте единый пайплайн предобработки с использованием ColumnTransformer:

- численные признаки: заполнение пропусков (например, медианой) + масштабирование (StandardScaler);
- категориальные признаки: заполнение пропусков (например, most_frequent) + кодирование (OneHotEncoder(handle_unknown='ignore')).

Пайплайн должен быть воспроизводимым и применяться одинаково к train и test.

[107]:
```
numeric_features = X.select_dtypes(include=["number"]).columns
categorical_features = X.select_dtypes(include=['object']).columns

print('numeric:', numeric_features)
print('categorical:', categorical_features)
```

```
numeric: Index(['Unnamed: 0', 'carat', 'depth', 'table', 'x', 'y', 'z'], dtype='str')
categorical: Index(['cut', 'color', 'clarity'], dtype='str')
```

[60]:
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

numeric_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

[61]:
```
categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])
```

[62]:
```
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer([
    ('num', numeric_pipeline, numeric_features),
    ('cat', categorical_pipeline, categorical_features)
])
```

[64]:
```
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

[65]:
```
print(X_train_processed.shape)
print(X_test_processed.shape)
```

```
(43152, 27)
(10788, 27)
```

4. Обучение базовых моделей

Обучите и сравните минимум 4 модели:

- LinearRegression (baseline);
- Ridge или Lasso (регуляризация);
- KNeighborsRegressor;
- DecisionTreeRegressor;

Дополнительно (рекомендуется): RandomForestRegressor или GradientBoostingRegressor.

[71]:
```
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

from sklearn.pipeline import Pipeline

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

import numpy as np
```

[73]:
```
models = {
    'LinearRegression': LinearRegression(),
    'Ridge': Ridge(),
    'KNN': KNeighborsRegressor(),
    'DecisionTree': DecisionTreeRegressor(random_state=42),
    'RandomForest':RandomForestRegressor(random_state=42)
}
```

[*]:
```
results = []
for name, model in models.items():
    pipe = Pipeline([
        ('preprocessing', preprocessor),
        ('model', model)
    ])
    pipe.fit(X_train, y_train)
    predictions = pipe.predict(X_test)

    mae = mean_absolute_error(y_test, predictions)
    rmse = np.sqrt(mean_squared_error(y_test, predictions))
    r2 = r2_score(y_test, predictions)

    results.append({
    "Model": name,
    "MAE": mae,
    "RMSE": rmse,
    "R2": r2,
    })
```

```
results_df = pd.DataFrame(result)
```

[*]:
```
results_df.sort_values("RMSE")
```

5. Подбор гиперпараметров

Выполните подбор гиперпараметров минимум для двух моделей (например, Ridge и KNN, или KNN и RandomForest) с использованием GridSearchCV либо RandomizedSearchCV. Используйте кросс-валидацию (cv=5).

[91]:
```
from sklearn.model_selection import GridSearchCV
```

[92]:
```
ridge_pipe = Pipeline([
    ('preprocessing', preprocessor),
    ('model', Ridge())
])
```

[95]:
```
ridge_params = {
    "model__alpha": [0.01, 0.1, 1, 10, 100]
}
```

[96]:
```
ridge_grid = GridSearchCV(
    ridge_pipe,
    ridge_params,
    cv=5,
    scoring="neg_root_mean_squared_error",
    n_jobs = -1
)

ridge_grid.fit(X_train,y_train)
```

[ ]:
```
ridge_grid.best_params_
```

[ ]:
```
knn_pipe = Pipeline([
    ('preprocessing', preprocessor),
    ('model', KNeighborsRegressor())
])
```

[ ]:
```
knn_params = {
    "model__n_neighbors": [3, 5, 7, 9],
    "model__weights": ["uniform", "distance"]
}
```

[ ]:
```
knn_grid = GridSearchCV(
    knn_pipe,
    knn_params,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs = -1
)
knn_grid.fit(X_train, y_train)
```

[ ]:
```
knn_grid.best_params_
```

6. Оценка качества

Для каждой модели посчитайте на тестовой выборке метрики:

- MAE
- RMSE
- R^2

Сведите результаты в единую таблицу (DataFrame) и отсортируйте по RMSE (по возрастанию).

[38]:
```
best_ridge = ridge_grid.best_estimator_
best_knn = knn_grid.best_estimator_
```

[39]:
```
ridge_pred = best_ridge.predict(X_test)
knn_pred = best_knn.predict(X_test)
```

[40]:
```
results = []
def evaluate(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    results.append({
    "model": name,
    "mae": mae,
    "rmse": rmse,
    "r2": r2
    })
evaluate("ridge (tunes)", y_test, ridge_pred)
evaluate("KNN (tunes)", y_test, knn_pred)
```

[41]:
```
results_df = pd.DataFrame(results)
results_df = results_df.sort_values("rmse")
results_df
```

[41]:

	model	mae	rmse	r2
1	KNN (tunes)	301.293185	641.408006	0.974120
0	ridge (tunes)	737.129785	1132.345198	0.919342

7. Визуализация результатов

Постройте графики для лучшей модели:

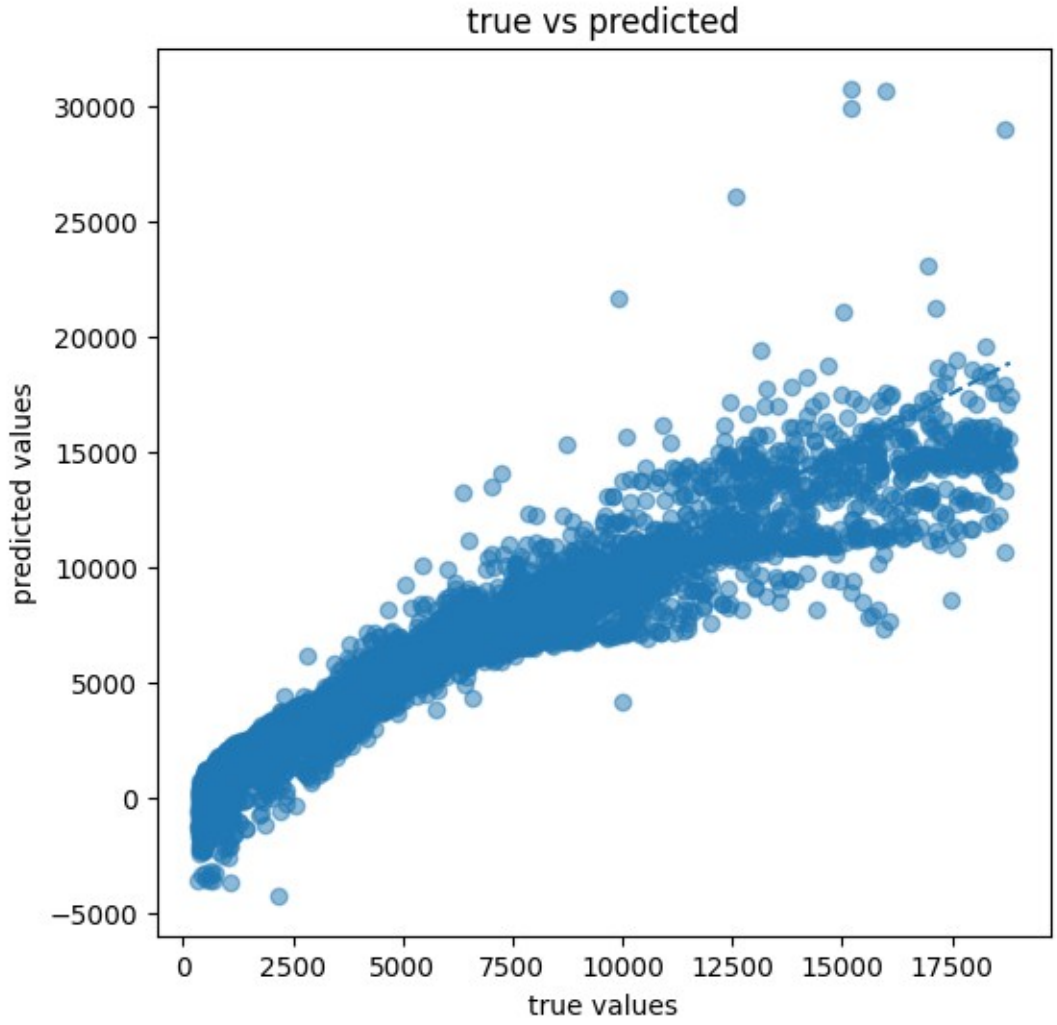
- точечная диаграмма y_true vs y_pred;
- гистограмма остатков (y_true - y_pred);
- график важности признаков/коэффициентов (если применимо: top -10).

[43]:
```
best_model = best_ridge
y_pred = best_model.predict(X_test)
residuals = y_test - y_pred
```

[45]:
```
plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel("true values")
plt.ylabel("predicted values")
plt.title("true vs predicted")

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle='--'
)

plt.show()
```



Вывод:

В рамках работы был реализован полный пайплайн решения задачи регрессии: выполнен анализ данных, построена воспроизводимая предобработка с использованием Pipeline и ColumnTransformer, обучены несколько моделей и проведён подбор гиперпараметров.
Лучший результат по метрике RMSE показала модель **KNN**. Наибольшее влияние на целевую переменную оказывают признаки **carat, clarity_I1, color, clarify_FI**. Выявленные проблемы данных (пропуски, различие масштабов, категориальные признаки) были корректно обработаны на этапе предобработки, что обеспечило устойчивость и воспроизводимость модели.